

The Alien Tiles Problem

CSPLib Problem 027

1	1	1	0
1	0	0	2
0	2	2	2
1	0	0	2

Bài toán tìm lời giải có tổng số lần click nhỏ nhất

Ví dụ trực quan, mô hình ILP, mô hình SAT và dữ liệu

Ngày cập nhật: 21 tháng 7 năm 2026

Tóm tắt nội dung

Tài liệu chỉ xét một mục tiêu: với trạng thái đích đã cho, tìm cách click có tổng số lần click nhỏ nhất. Trạng thái ban đầu được lấy là lưới toàn 0, đúng theo quy ước của CSPLib prob027. Trước tiên, một ví dụ 4×4 với ba màu được giải từng bước. Sau đó cùng ví dụ này được dùng để giải thích mô hình quy hoạch tuyến tính nguyên (ILP) và mô hình SAT có cận chi phí. Phần cuối thiết kế một quy trình sinh benchmark tái lập, mô tả các file dữ liệu trong dự án và dẫn nguồn paper/web.

Bài toán trong một câu

Đầu vào: kích thước N , số màu c và target T .

Đầu ra: số lần click x_{ij} tại mỗi ô.

Mục tiêu duy nhất:

$$\min \sum_{i=1}^N \sum_{j=1}^N x_{ij},$$

với điều kiện các click phải biến lưới ban đầu toàn 0 thành đúng target T .

Mục lục

1	Luật chơi và dữ liệu đầu vào	1
1.1	Dạng instance dùng trong tài liệu	1
2	Ví dụ xuyên suốt: optimum bằng 3 click	1
3	Mô hình toán học chung	2
4	Mô hình ILP	3
4.1	Mô hình đầy đủ	3
4.2	Đọc biến phụ q bằng ví dụ	4
5	Mô hình SAT	4
5.1	Bước 1: mã hóa số lần click ở mỗi ô	4
5.2	Bước 2: cộng các click theo modulo	5
5.3	Bước 3: mã hóa cận tổng số click	5
5.4	Tìm giá trị tối ưu	6
6	Dataset và nguồn tham khảo	6
6.1	Nguồn chính	6
6.2	Mục tiêu thiết kế benchmark	7
6.3	Hai họ instance bổ sung nhau	7
6.4	Phân tầng easy, medium và hard	8
6.5	Nhãn và quy trình sinh	9
6.6	Triển khai trong dự án	9
6.7	Các instance kiểm chứng thủ công	10
6.8	Schema dữ liệu sinh tự động	10
7	Cách chạy mô hình SAT đi kèm	11
8	Kết luận	11

1 Luật chơi và dữ liệu đầu vào

Ta có một lưới vuông $N \times N$. Mỗi ô nhận một số trong

$$\{0, 1, \dots, c-1\}.$$

Các số này đại diện cho c màu nằm trên một vòng tròn. Sau màu $c-1$ ta quay lại màu 0.

Khi click ô (i, j) một lần:

- mọi ô trên hàng i tăng thêm 1;
- mọi ô trên cột j tăng thêm 1;
- ô (i, j) cũng chỉ tăng 1, không phải tăng 2;
- tất cả phép tăng đều được lấy modulo c .

Ví dụ với $c = 3$, màu thay đổi theo chu trình

$$0 \longrightarrow 1 \longrightarrow 2 \longrightarrow 0.$$

Thứ tự click không quan trọng. Vì vậy, thay vì lưu một chuỗi thao tác, ta chỉ cần lưu

$$x_{ij} = \text{số lần click ô } (i, j).$$

Do click c lần tại cùng một ô không tạo thay đổi ròng, chỉ cần xét

$$x_{ij} \in \{0, 1, \dots, c-1\}.$$

Các tính chất này xuất phát trực tiếp từ đặc tả CSPLib [2, 6].

1.1 Dạng instance dùng trong tài liệu

Một instance gồm ba thành phần:

$$(N, c, T),$$

trong đó $T = (t_{rk})$ là ma trận target. Trạng thái đầu là ma trận toàn 0. Nếu cần trạng thái đầu S khác 0, có thể thay target bằng

$$d_{rk} = (t_{rk} - s_{rk}) \bmod c$$

rồi giải như trường hợp bắt đầu từ 0.

2 Ví dụ xuyên suốt: optimum bằng 3 click

Xét lưới 4×4 , $c = 3$ và target

$$T = \begin{bmatrix} 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 2 \\ 0 & 2 & 2 & 2 \\ 1 & 0 & 0 & 2 \end{bmatrix}. \quad (1)$$

Một lời giải là:

- click ô $(1, 1)$ đúng 1 lần;
- click ô $(3, 4)$ đúng 2 lần;

- không click các ô còn lại.

Ma trận click tương ứng là

$$X = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 2 \\ 0 & 0 & 0 & 0 \end{bmatrix}. \quad (2)$$

Kiểm tra một vài ô

Ô (1, 2): cùng hàng với (1, 1) nên nhận 1; không cùng hàng/cột với (3, 4). Kết quả là 1.

Ô (2, 4): cùng cột với (3, 4) nên nhận 2. Kết quả là 2.

Ô (1, 4): nhận 1 từ (1, 1) và nhận 2 từ (3, 4):

$$(1 + 2) \bmod 3 = 0.$$

Ô (3, 1): nhận 2 từ (3, 4) và nhận 1 từ (1, 1):

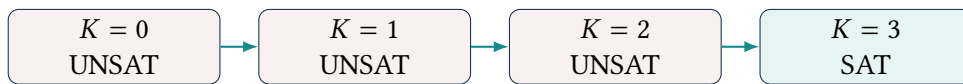
$$(2 + 1) \bmod 3 = 0.$$

Các giá trị này khớp với target trong phương trình (1).

Tổng số click của phương trình (2) là

$$1 + 2 = 3.$$

Đây chưa đủ để kết luận optimum bằng 3: ta còn phải chứng minh không có lời giải nào dùng 0, 1 hoặc 2 click. Bộ giải SAT đi kèm đã kiểm tra các cận này:



Do đó giá trị đầu tiên có lời giải là $K = 3$, nên optimum bằng 3.

Tại sao phải tối thiểu hóa?

Một target có thể có nhiều ma trận click khác nhau. Chẳng hạn, với $N = 4, c = 3$, click mỗi ô hàng 1 một lần và mỗi ô hàng 2 hai lần sẽ không làm thay đổi lưới sau khi lấy modulo 3. Target toàn 0 vì vậy có cả lời giải 0 click và một lời giải 12 click. Hàm mục tiêu phải chọn lời giải 0 click.

3 Mô hình toán học chung

Đặt tập chỉ số $\mathcal{J} = \{1, \dots, N\}$. Biến quyết định là

$$x_{ij} \in \{0, 1, \dots, c-1\}, \quad i, j \in \mathcal{J}.$$

Xét ô target (r, k) . Tổng số lần ô này bị tác động là

$$a_{rk} = \sum_{j \in \mathcal{J}} x_{rj} + \sum_{i \in \mathcal{J}} x_{ik} - x_{rk}. \quad (3)$$

Cách đọc công thức:

1. tổng thứ nhất đếm mọi click trên hàng r ;
2. tổng thứ hai đếm mọi click trên cột k ;
3. x_{rk} bị đếm hai lần, nên phải trừ đi một lần.

Để ô (r, k) có đúng màu target t_{rk} , ta cần

$$a_{rk} \bmod c = t_{rk}, \quad \forall r, k \in \mathcal{J}. \quad (4)$$

Mô hình tối ưu ở dạng modulo

$$\min \sum_{i \in \mathcal{J}} \sum_{j \in \mathcal{J}} x_{ij} \quad (5)$$

$$\text{s.t.} \quad \left(\sum_{j \in \mathcal{J}} x_{rj} + \sum_{i \in \mathcal{J}} x_{ik} - x_{rk} \right) \bmod c = t_{rk}, \quad \forall r, k \in \mathcal{J}, \quad (6)$$

$$x_{ij} \in \{0, 1, \dots, c-1\}, \quad \forall i, j \in \mathcal{J}. \quad (7)$$

Phương trình (5) là hàm mục tiêu duy nhất của tài liệu: tổng số click càng nhỏ càng tốt. Phương trình (6) bảo đảm lời giải tạo đúng target.

4 Mô hình ILP

ILP không dùng trực tiếp toán tử modulo. Ta thay mỗi điều kiện

$$a_{rk} \bmod c = t_{rk}$$

bằng một phương trình nguyên

$$a_{rk} - cq_{rk} = t_{rk}. \quad (8)$$

q_{rk} chỉ là phần thương khi chia a_{rk} cho c ; nó không phải số lần click. Từ

$$0 \leq a_{rk} \leq (2N-1)(c-1),$$

ta có thể dùng cận an toàn

$$0 \leq q_{rk} \leq \left\lfloor \frac{(2N-1)(c-1)}{c} \right\rfloor.$$

4.1 Mô hình đầy đủ

ILP tối thiểu hóa số lần click

$$\min \sum_{i \in \mathcal{J}} \sum_{j \in \mathcal{J}} x_{ij} \quad (9)$$

$$\text{s.t.} \quad \sum_{j \in \mathcal{J}} x_{rj} + \sum_{i \in \mathcal{J}} x_{ik} - x_{rk} - cq_{rk} = t_{rk}, \quad \forall r, k \in \mathcal{J}, \quad (10)$$

$$x_{ij} \in \mathbb{Z}, \quad 0 \leq x_{ij} \leq c-1, \quad \forall i, j \in \mathcal{J}, \quad (11)$$

$$q_{rk} \in \mathbb{Z}, \quad 0 \leq q_{rk} \leq \left\lfloor \frac{(2N-1)(c-1)}{c} \right\rfloor, \quad \forall r, k \in \mathcal{J}. \quad (12)$$

Mô hình có N^2 biến click x và N^2 biến phụ q . Có đúng N^2 phương trình tác động, mỗi phương trình ứng với một ô target. Đây là dạng tổng quát của mô hình IP tối thiểu số click được Chlond trình bày cho Alien Tiles 7×7 với bốn màu [1].

4.2 Đọc biến phụ q bằng ví dụ

Quay lại target ở phương trình (1).

Tại ô (1, 4):

$$\underbrace{\sum_j x_{1j}}_{=1} + \underbrace{\sum_i x_{i4}}_{=2} - \underbrace{x_{14}}_{=0} - 3q_{14} = \underbrace{t_{14}}_{=0}.$$

Vế tác động trước modulo là 3, nên $q_{14} = 1$ và $3 - 3 \cdot 1 = 0$.

Tại ô (1, 1):

$$\underbrace{\sum_j x_{1j}}_{=1} + \underbrace{\sum_i x_{i1}}_{=1} - \underbrace{x_{11}}_{=1} - 3q_{11} = \underbrace{t_{11}}_{=1}.$$

Ở đây tác động đã bằng target, nên $q_{11} = 0$. Hai phép tính này cho thấy q chỉ giúp ILP bỏ đi một số bội của c .

5 Mô hình SAT

SAT cơ bản chỉ trả lời có hoặc không. Vì vậy ta đưa vào một cận K và hỏi:

Có tồn tại lời giải tạo đúng target và dùng tổng cộng không quá K click hay không?

Gọi công thức này là Φ_K . Giá trị tối ưu là cận K nhỏ nhất làm Φ_K khả thỏa.

5.1 Bước 1: mã hóa số lần click ở mỗi ô

Với mỗi ô (i, j) và mỗi $v \in \{0, \dots, c-1\}$, tạo biến Boolean

$$d_{ijv} = 1 \iff x_{ij} = v.$$

Ví dụ, khi $c = 3$, ô (i, j) có ba biến

$$d_{ij0}, \quad d_{ij1}, \quad d_{ij2}.$$

Đúng một trong ba biến phải nhận true. Mã hóa CNF theo cặp là

$$d_{ij0} \vee d_{ij1} \vee \dots \vee d_{ij,c-1}, \tag{13}$$

$$\neg d_{ija} \vee \neg d_{ijb}, \quad 0 \leq a < b < c. \tag{14}$$

Phương trình (13) nói rằng phải chọn ít nhất một giá trị. Phương trình (14) cấm chọn đồng thời hai giá trị.

5.2 Bước 2: cộng các click theo modulo

Đối với ô target (r, k) , liệt kê $L = 2N - 1$ biến click tác động lên nó:

$$P_{rk} = (x_{r1}, \dots, x_{rN}, x_{1k}, \dots, x_{r-1,k}, x_{r+1,k}, \dots, x_{Nk}).$$

Biến x_{rk} chỉ xuất hiện một lần trong danh sách.

Tạo biến Boolean

$$s_{rk,\ell,a} = 1$$

với ý nghĩa: sau khi cộng ℓ phần tử đầu của P_{rk} , phần dư hiện tại là a . Tại mỗi bước ℓ , đúng một trong

$$s_{rk,\ell,0}, \dots, s_{rk,\ell,c-1}$$

được chọn.

Trạng thái đầu là phần dư 0:

$$s_{rk,0,0}. \quad (15)$$

Giả sử phần dư cũ là a và click tiếp theo có giá trị b . Phần dư mới phải là $(a + b) \bmod c$. Nếu click tiếp theo là x_{uv} , clause chuyển là

$$\neg s_{rk,\ell-1,a} \vee \neg d_{uvb} \vee s_{rk,\ell,(a+b) \bmod c}. \quad (16)$$

Với $c = 3$, quy tắc chuyển có thể đọc trực tiếp từ bảng:

Bảng 1: Phần dư mới $(a + b) \bmod 3$.

$a \backslash b$	0	1	2
0	0	1	2
1	1	2	0
2	2	0	1

Sau L bước, phần dư phải bằng target:

$$s_{rk,L,t_{rk}}, \quad \forall r, k \in \mathcal{J}. \quad (17)$$

5.3 Bước 3: mã hóa cận tổng số click

Ta cần chuyển giá trị x_{ij} thành các đóng góp đơn vị. Với mỗi $h = 1, \dots, c - 1$, tạo

$$u_{ijh} = 1 \iff x_{ij} \geq h. \quad (18)$$

Liên kết với các biến one-hot ở bước 1 bằng

$$u_{ijh} \iff \bigvee_{v=h}^{c-1} d_{ijv}. \quad (19)$$

Với $c = 3$, ta có bảng minh họa:

x_{ij}	u_{ij1}	u_{ij2}	Tổng đóng góp
0	0	0	0
1	1	0	1
2	1	1	2

Do đó

$$\sum_{i,j} x_{ij} = \sum_{i,j} \sum_{h=1}^{c-1} u_{ijh}. \quad (20)$$

Cận tổng số click không quá K trở thành cardinality constraint

$$\text{AtMost}(\{u_{ijh}\}, K). \quad (21)$$

Ràng buộc phương trình (21) có thể chuyển sang CNF bằng sequential counter [7]. Công thức SAT đầy đủ Φ_K gồm:

1. exactly-one cho các biến d_i ;
2. các trạng thái cộng modulo và clause chuyển;
3. trạng thái cuối bằng target;
4. cardinality constraint tổng số click không quá K .

5.4 Tìm giá trị tối ưu

Cách dễ hiểu nhất là thử $K = 0, 1, 2, \dots$ cho đến khi SAT. Với ví dụ xuyên suốt:

K	0	1	2	3
Φ_K	UNSAT	UNSAT	UNSAT	SAT

Vì $K = 3$ là cận nhỏ nhất khả thỏa, lời giải có tổng click bằng 3 là tối ưu. Khi miền cận lớn, có thể dùng tìm kiếm nhị phân hoặc giữ một solver SAT incremental để tái sử dụng các clause đã học.

6 Dataset và nguồn tham khảo

6.1 Nguồn chính

- CSPLib prob027 cung cấp luật chơi và các mô hình Essence [2, 4].
- Chlond (2006) trình bày trực tiếp mô hình IP tối thiểu hóa số click cho Alien Tiles 7×7 , bốn màu [1].
- Gent, Linton và Smith (2000) công bố một ma trận click 4×4 , $c = 3$ có tổng 10; target suy ra từ ma trận này được dùng như một instance cố định trong bộ dữ liệu [6, 5].

Trang Data files của CSPLib hiện không liệt kê file instance nào [3]. Vì vậy các file JSON trong dự án là bộ dữ liệu thực nghiệm đi kèm, không phải bộ dữ liệu chính thức của CSPLib.

6.2 Mục tiêu thiết kế benchmark

Một bộ benchmark dùng cho khóa luận cần đáp ứng đồng thời bốn yêu cầu:

1. có instance chắc chắn có lời giải để đo khả năng tối ưu hóa;
2. có target không bị ép phải có lời giải, nhờ đó vẫn đánh giá được khả năng phát hiện UNSAT;
3. phủ nhiều giá trị N, c và nhiều mức độ thừa/dày;
4. có thể sinh lại đúng từng byte từ cấu hình và seed đã công bố.

Điểm dễ gây nhầm lẫn

Nếu target được sinh từ ma trận click X^w có tổng U , ta chỉ biết optimum không vượt quá U . Có thể tồn tại ma trận click khác có tổng nhỏ hơn. Vì vậy generator ghi `known_upper_bound = U`, không ghi `known_optimum = U`.

6.3 Hai họ instance bổ sung nhau

6.3.1 Họ witness: có lời giải theo cách xây dựng

Với mỗi bộ tham số (N, c, ρ) , đặt

$$s = \max(1, \text{round}(\rho N^2)).$$

Generator chọn đều s ô khác nhau. Tại mỗi ô được chọn, số click được lấy đều trong $\{1, \dots, c-1\}$; các ô còn lại bằng 0. Ma trận nhận được gọi là witness X^w . Target được tính theo đúng luật chơi:

$$T_{rk} = \left(\sum_j x_{rj}^w + \sum_i x_{ik}^w - x_{rk}^w \right) \bmod c. \quad (22)$$

Instance này chắc chắn khả thi vì X^w tạo ra T . Tham số ρ điều khiển tỷ lệ ô click khác 0 trong witness, chủ không trực tiếp điều khiển độ thừa của target.

6.3.2 Họ uniform-target: chưa biết SAT hay UNSAT

Mỗi phần tử target được lấy độc lập và đều:

$$T_{ij} \sim \text{Uniform}\{0, 1, \dots, c-1\}. \quad (23)$$

Generator không tạo witness và gán trạng thái `unclassified`. Sau đó solver chính xác mới phân loại instance thành `OPTIMAL`, `UNSAT` hoặc `TIMEOUT`.

Bảng 2: Vai trò của hai họ instance.

Họ	Biết ngay khi sinh	Dùng để đánh giá
Witness	Có lời giải; tổng click của witness là upper bound	Khả năng tìm và chứng minh nghiệm tối ưu trên các mật độ click khác nhau
Uniform-target	Chỉ biết target hợp lệ về định dạng	Cả tối ưu hóa trên instance khả thi và phát hiện instance không có lời giải

Chỉ dùng họ witness sẽ làm bộ dữ liệu lệch hoàn toàn về phía instance khả thi. Chỉ dùng họ uniform-target lại không bảo đảm có đủ instance khả thi ở từng cấu hình. Hai họ vì vậy phải được giữ riêng trong báo cáo kết quả.

6.4 Phân tầng easy, medium và hard

Kích thước mô hình tăng đồng thời theo N và c . Generator dùng chỉ số cấu trúc

$$B(N, c) = N^2(c - 1). \quad (24)$$

Đây chính là số biến đóng góp đơn vị u_{ijh} dùng để đếm tổng click trong mô hình SAT ở phương trình (20). Generator sắp xếp tất cả các cặp (N, c) theo B , sau đó chia danh sách thành ba phần bằng nhau. Với cấu hình mặc định, sáu cặp thuộc mỗi nhóm như sau:

Bảng 3: Phân nhóm theo kích thước cấu trúc $B = N^2(c - 1)$.

Nhóm	Các cặp (N, c) theo thứ tự tăng của B	Khoảng B
Easy	(4, 2), (5, 2), (4, 3), (6, 2), (4, 4), (5, 3)	16–50
Medium	(8, 2), (6, 3), (5, 4), (10, 2), (6, 4), (8, 3)	64–128
Hard	(12, 2), (8, 4), (10, 3), (12, 3), (10, 4), (12, 4)	144–432

Cách hiểu đúng nhãn độ khó

Easy, medium và hard ở đây là nhãn *kích thước cấu trúc*, được xác định trước khi chạy solver. Runtime thực tế còn phụ thuộc target, optimum và trạng thái SAT/UNSAT. Sau thực nghiệm có thể bổ sung `observed_difficulty`, nhưng không thay đổi nhãn cấu trúc ban đầu.

Cấu hình benchmark đầy đủ là:

Bảng 4: Cấu hình benchmark khuyến nghị cho thực nghiệm chính.

Thành phần	Cách lấy mẫu	Số instance
Witness cơ sở	Một instance cho mỗi (N, c, ρ) , với $N \in \{4, 5, 6, 8, 10, 12\}$, $c \in \{2, 3, 4\}$ và $\rho \in \{0.10, 0.30, 0.60\}$	54
Witness bổ sung	Hai instance thêm cho mỗi nhóm độ khó; mật độ được luân phiên	6
Uniform-target	Hai instance cho mỗi nhóm độ khó; giá trị c được luân phiên	6

Phần witness trước hết phủ toàn bộ các tổ hợp (N, c, ρ) :

$$6 \times 3 \times 3 = 54.$$

Sáu witness bổ sung làm tổng witness bằng 60. Sáu uniform-target được chia đều hai instance cho mỗi nhóm. Tổng kích thước benchmark là

$$60 + 6 = 66.$$

Mỗi nhóm easy, medium và hard có 20 witness và 2 uniform-target, tức 22 instance. Cách phân bổ này giữ được độ phủ của witness nhưng tránh tạo hàng trăm file gần giống nhau.

Để việc sinh dữ liệu không phụ thuộc thứ tự vòng lặp, mỗi instance dùng một seed 64-bit riêng:

$$\text{seed}_{\mathcal{J}} = \text{first64}(\text{SHA256}(S, N, c, \text{họ}, \rho, r, a)),$$

trong đó S là master seed, r là số thứ tự lặp và a là số lần thử lại khi gặp target trùng. Target được nhận diện bằng SHA-256 của bộ (N, c, T) . Nếu hash đã xuất hiện trong bộ dữ liệu thì generator tăng a và sinh lại. Các phép lấy mẫu tiếp theo dùng bộ sinh counter-mode SHA-256 do dự án cài đặt, vì vậy kết quả không phụ thuộc vào thay đổi nội bộ của random. Random giữa các phiên bản Python.

6.5 Nhãn và quy trình sinh

Mỗi instance đi qua các bước sau:

1. đọc một tầng (N, c, h_0, ρ, r) và tính $B = N^2(c - 1)$;
2. dẫn xuất instance seed từ master seed;
3. sinh witness hoặc sinh trực tiếp target;
4. tính target, thống kê tần suất màu và hash;
5. loại target trùng trong cùng bộ benchmark;
6. kiểm tra kích thước, miền giá trị và kiểm tra lại witness;
7. ghi JSON, manifest.csv và dataset_summary.json.

Quy tắc gắn nhãn được cố định như sau:

- witness: feasible_by_construction và có known_upper_bound;
- uniform-target: unclassified;
- solver tìm được lời giải K click nhưng chưa chứng minh tối ưu: chỉ cập nhật upper bound;
- solver chứng minh tối ưu: mới ghi known_optimum = K ;
- solver chứng minh vô nghiệm: ghi known_status = UNSAT;
- hết thời gian: ghi TIMEOUT, không được đổi thành UNSAT.

6.6 Triển khai trong dự án

Generator chỉ dùng thư viện chuẩn Python và từ chối ghi đè thư mục đã tồn tại. Bộ 66 instance nằm trong data/benchmark_v1 và được sinh bằng:

```
python3 experiments/generate_instances.py \
--output-dir data/benchmark_v1 \
--sizes 4 5 6 8 10 12 --colours 2 3 4 \
--densities 0.10 0.30 0.60 \
--witness-count 60 --uniform-count 6 \
--master-seed 270027
```

Kiểm tra toàn bộ dataset bằng:

```
python3 experiments/validate_dataset.py data/benchmark_v1
```

Các file được tổ chức theo size_tier/family/nNN_cCC/. File manifest.csv là chỉ mục dùng cho chương trình chạy thực nghiệm; dataset_summary.json lưu toàn bộ cấu hình để có thể tái lập.

6.7 Các instance kiểm chứng thủ công

Bảng 5: Bộ dữ liệu dùng để thử mô hình tối thiểu số click.

File	N	c	Mô tả	Kết quả
3x3_c2_allones.json	3	2	Target toàn 1	minimum 3
4x4_c3_allones.json	4	3	Target toàn 1	minimum 4
4x4_c3_easy.json	4	3	Ví dụ xuyên suốt ở mục 2	minimum 3
4x4_c3_infeasible.json	4	3	Instance âm để kiểm tra solver	không có lời giải
4x4_c3_hardest_gent2000.json	4	3	Target suy ra từ witness đã công bố	minimum 10 [6]
7x7_c4_solvable.json	7	4	Target tự sinh từ witness 6 click	upper bound 6

Các giá trị minimum 3, 4 và 10 đã được kiểm tra bằng `models/sat_variant2.py`. Với file 7×7 , dự án mới gần nhãn upper bound 6; không nên gọi 6 là optimum nếu chưa chứng minh không tồn tại lời giải nhỏ hơn.

6.8 Schema dữ liệu sinh tự động

Một instance witness sinh tự động có các trường chính sau:

Listing 1: Schema JSON rút gọn của một instance witness.

```
{
  "schema_version": "2.0",
  "name": "easy_n04_c03_witness_d100_r000",
  "N": 4,
  "c": 3,
  "target": [[...], [...], [...], [...]],
  "generation": {
    "method": "witness",
    "size_tier": "easy",
    "structural_score": 32,
    "master_seed": 270027,
    "instance_seed": 123456789,
    "target_sha256": "...",
  },
  "guaranteed_solvable": true,
  "status_at_generation": "feasible_by_construction",
  "witness_clicks": [[...], [...], [...], [...]],
  "witness_total_clicks": 5,
  "known_upper_bound": 5
}
```

Instance uniform-target không có ba trường witness và upper bound ở cuối. Trường `known_optimum` cũng không xuất hiện khi sinh; nó chỉ được bổ sung sau khi một solver chính xác đã chứng minh giá trị tối ưu.

7 Cách chạy mô hình SAT đi kèm

Script tối ưu target cố định là `models/sat_variant2.py`. Chạy ví dụ xuyên suốt bằng:

```
python3 models/sat_variant2.py \
--input data/4x4_c3_easy.json
```

Thêm số `--input-dir` đọc đệ quy các file instance và tự bỏ qua `dataset_summary.json`. Ví dụ chạy toàn bộ nhóm easy:

```
python3 models/sat_variant2.py \
--input-dir data/benchmark_v1/easy
```

Kết quả cần kiểm tra gồm:

- `Optimal X`: ma trận số lần click;
- `Minimum total clicks`: giá trị hàm mục tiêu;
- `Verification: PASSED`: áp dụng lại ma trận click tạo đúng target.

Với instance không có lời giải, script trả về `UNSATISFIABLE`; khi đó không tồn tại giá trị minimum.

8 Kết luận

Bài toán được nghiên cứu trong tài liệu có một mục tiêu rõ ràng:

$$\min \sum_{i,j} x_{ij}.$$

Các ràng buộc hàng, cột và modulo bảo đảm ma trận click tạo đúng target. Trong ILP, modulo được thay bằng biến thương nguyên q_{rk} . Trong SAT, ta hỏi lần lượt liệu có lời giải với tổng click không quá K hay không; cận K nhỏ nhất trả về SAT chính là optimum.

Ví dụ 4×4 , $c = 3$ cho thấy toàn bộ quá trình: ma trận click có tổng 3 tạo đúng target, còn các cận 0, 1 và 2 đều UNSAT. Vì vậy minimum bằng 3.

Tài liệu

- [1] Martin J. Chlond. An alien IP. *INFORMS Transactions on Education*, 7(1):149–152, 2006. URL: <https://pubsonline.informs.org/doi/10.1287/ited.7.1.149>, doi:10.1287/ited.7.1.149.
- [2] CSPLib. Problem 027: Alien tiles problem. <https://www.csplib.org/Problems/prob027/>, 2026. Truy cập ngày 21-07-2026.
- [3] CSPLib. Problem 027: Data files. <https://www.csplib.org/Problems/prob027/data/>, 2026. Truy cập ngày 21-07-2026.
- [4] CSPLib. Problem 027: Models. <https://www.csplib.org/Problems/prob027/models/>, 2026. Truy cập ngày 21-07-2026.
- [5] CSPLib. Problem 027: Results. <https://www.csplib.org/Problems/prob027/results/>, 2026. Truy cập ngày 21-07-2026.
- [6] Ian P. Gent, Steve A. Linton, and Barbara M. Smith. Symmetry breaking in the alien tiles puzzle. Technical Report APES-22-2000, APES Research Group, October 2000. Bản sao PDF được lưu trong thư mục references của dự án. URL: <https://www.csplib.org/Problems/prob027/references/>.
- [7] Carsten Sinz. Towards an optimal CNF encoding of boolean cardinality constraints. In *Principles and Practice of Constraint Programming – CP 2005*, volume 3709 of *Lecture Notes in Computer Science*, pages 827–831. Springer, 2005. doi:10.1007/11564751_73.